SAARTHI — Design Document v2

Offline-first agentic learning companion for regional-language learners · Team Titanic Swimming · NEXTECH 2.0, NMIET · Revision 2, Oct 11, 2026

0. What changed in revision 2

This revision applies the architecture change checklist to the v1 design. The sound parts of v1 are kept (deterministic core, offline-first PWA, event sourcing, hybrid retrieval). The changes fix contradictions, add missing contracts, and turn targets into measurable acceptance criteria. Everything here is a proposed design, not a claim about what is already built.

Where each checklist item is answered

Checklist items	Change	Section
1	Voice scope resolved: text-first Odia doubts are P0; online voice is P1; offline learning never needs ASR or TTS	3, 12, 17
2, 16, 17	Stateless compute versus durable state, write ownership, storage roles, transaction boundary, one worker framework with a durable job table	5
3	Device and profile separated; profile isolation rules and tests	7, 15
4, 10	Per-event sync outcomes, outbox and quarantine rules, auth renewal after long offline periods	8
5, 7	Canonical ordering and replay policy, single device per learner for the MVP, mastery transition policy per event type, BKT rules	6, 9
6	Action rules and guard corrected, terminal behaviours defined, reason codes stable	6, Appendix A
8	Reproducible decision versus auditable (not identical) generated text, generation records	4, 7

Checklist items	Change	Section
9	Ordered validation contract, deterministic versus model checks	13
11, 29	Release gates for children's data and content rights; honest maturity statement	1, 14
12, 27	Full agent loop diagram, bounded autonomy, no LLM in the core loop	5, 6
13, 14	Capability router, fallback matrix, four network paths, non-blocking reconnect	5, 10
15	Content flow end to end, stable IDs across versions	11
18 to 22	Decision-quality metric, measurable hardware targets, cross-language parity, expanded fault matrix, Odia release gate	15
23 to 26, 28	MVP kept small, contracts frozen first, dependency-ordered milestones, done means verified, no extra infrastructure	2, 16

Decisions made in this revision that the team should confirm

- Correctness is scoped to one device per learner in the MVP. Several learners may share one phone; one learner may not sync from two phones until a merge policy is tested (Section 9).
- The milestone order follows the checklist, so it replaces the five-week plan in the pitch deck. The deck's implementation slide and success-metric slide should be updated to match (Section 16).
- Arq is the worker framework, with a PostgreSQL job table as the source of truth (Section 5).
- Mastery is stored as integer basis points so Python and TypeScript decisions match exactly (Section 6).

1. Purpose, rules and maturity

This is the single source of truth for building SAARTHI. Every teammate and every coding agent works from it, and a decision made here is binding until someone edits it here.

Product in one sentence: SAARTHI is a stateful learning agent that decides a Class 9 Odia-medium learner's next educational action from what they know, what they need and what the device can currently do, and keeps working when the network drops.

Maturity statement (Decided). SAARTHI is built with production engineering practice, but until reliability, privacy, security, Odia quality and target-device performance have each been verified by the tests in Section 15, it is described as a production-minded, offline-first MVP. The design specifying backups, monitoring, TLS and retries does not make it production-ready; running the operational tests and publishing the results does.

Rules for humans and coding agents

- Build only what this document specifies. If a detail is missing, pick the stated default, record it in `docs/decisions.md`, and add it to Section 18.
- The rule engine chooses the learning action. The LLM writes words and never picks the action (Section 4).
- Offline is the default path, not an error path. Every feature ships with its offline behaviour defined and tested.
- Every tool and provider call returns a typed result or a typed error. No unhandled exception crosses a service boundary.
- Compute is stateless; correctness never depends on process memory (Section 5).
- Contracts are generated: Pydantic models are the source, OpenAPI and TypeScript types are generated from them. Never hand-edit generated types, and never change a contract in only one service.
- A feature is done only when it meets the Definition of Done in Section 16, which requires verified behaviour, not a component that merely starts.
- All learner-facing text is Odia-first. English strings are developer fallbacks only.

Instructions to a coding agent: read Sections 1 to 9 and Appendix A fully before writing code, implement in the milestone order of Section 16, and after each milestone run the tests of Section 15 and report results against that milestone's gate.

Labels used: *Decided* means build it this way. *Default* means build it this way unless the owner changes it. *Open* means unresolved and tracked in Section 18.

2. Problem, learner and scope

The gap in one sentence: a Class 9 learner in rural or semi-urban Odisha, on a shared low-end phone with weak signal, has no tool that understands her doubts in Odia, tracks what she has not understood, and still works when the network drops.

Target learner (reference persona): Sunita, Class 9, government school, learns in Odia, meets English mostly through textbook terms. One shared smartphone for about an hour each evening, limited data pack, signal that comes and goes. Her teacher cannot answer every doubt.

Need	SAARTHI capability	Section
Ask a doubt in Odia and get an answer at her level	Text doubt flow with grounded retrieval and Odia generation; online voice as an optional input	6, 11, 12
Keep learning when the signal drops, resume where she left off	Offline-first PWA, local state, event queue, sync	9
Be told what to study next without asking	Rule-based next-action engine over mastery state	6
Use it without training on a shared phone	Text-first, minimal screens, several profiles on one phone	3, 14

MVP scope (Decided): Class 9, Science, Odia. Three chapters with full coverage (concept notes, practice questions, misconception tags) rather than the whole textbook shallowly.

Critical path (must work, in this order of priority): text-based Odia doubts, curated assessment, mastery update, deterministic next action, decision trace viewer, offline packs, reliable sync.

Out of the critical path until every P0 test passes: online voice, photo or diagram input, other subjects and languages, native Android app, on-device language models, teacher dashboard, a learner using two phones.

Later (Default order): more subjects, more Indic languages (Kui and other tribal languages need dedicated content work), multiple grades, teacher support view, multi-device learners.

Non-goal: SAARTHI is not a general chatbot. Off-curriculum questions are politely redirected, not answered freely.

Infrastructure rule (Decided): add a component only when a defined requirement, a measured bottleneck or an operational need justifies it. The MVP stack in Section 14 is sufficient if its boundaries and failure behaviour are implemented correctly; Temporal, Kafka and extra databases are not added.

3. Requirements

Each requirement has an ID so tests, tickets and commits can reference it. P0 must work in the final demo, P1 should, P2 is stretch.

Voice scope (Decided, resolves the v1 contradiction). Text-first Odia doubts are P0. Online Odia voice input and output (ASR and TTS) is P1, always with text as the fallback. Offline learning never depends on ASR or TTS: offline, the learner reads, answers by tapping and types doubts, and gets a pack-based answer. The demo script, milestone plan and acceptance tests in Sections 15 to 17 all follow this scope.

Functional requirements

ID	Requirement	Priority
F1	Learner picks a profile (several per phone) with an optional local 4-digit PIN; language defaults to Odia	P0
F2	Learner types a doubt in Odia (script, Latin-script or code-mixed) and receives an Odia answer grounded in curriculum chunks, with chapter reference and chunk IDs shown in the trace	P0
F3	Learner takes short assessments (MCQ first); each graded answer updates per-concept mastery under the transition policy of Section 6	P0
F4	After every learning event the agent returns one learning action (Remediate, Retry, Revise or Advance), a feasible delivery mode, any background actions (Sync, Prefetch) and a one-line Odia reason	P0
F5	With the network off, the learner continues with installed packs, takes local assessments and receives local decisions	P0

ID	Requirement	Priority
F6	Offline events are queued and synced on reconnect with no loss and no duplicates; every event gets an explicit outcome (Section 8)	P0
F7	Profiles on one phone are fully isolated in local state, outbox, decisions and cached UI	P0
F8	Online Odia voice input (ASR) and spoken output (TTS) with text fallback	P1
F9	Agent prefetches the next pack on a good connection, within the device storage budget	P1
F10	Decision trace viewer shows tools called, order, reason code and ruleset version	P0
F11	A learner whose auth has expired during a long offline period keeps learning and recovers without losing pending events (Section 8)	P0
F12	Photo or diagram input	P2

Non-functional requirements. All numbers are targets to be verified on the reference device of Section 15, not established capabilities.

ID	Requirement	Target to verify
N1	First meaningful screen on the throttled network profile	Under 4 s from cache, under 8 s cold
N2	Installed app plus one chapter pack	Under 25 MB; if measurement shows it is infeasible, change the pack strategy (Section 15)
N3	Local decision latency	Under 150 ms
N4	Online text doubt answer	p50 under 4 s, p95 under 9 s
N5	Offline events lost across any fault in the matrix	0
N6	Memory on the 2 GB reference device	No crash in a 20-minute session

ID	Requirement	Target to verify
N7	Every rule-based decision reproducible from recorded state, environment snapshot and ruleset version	100%
N8	Learner data minimal: no phone number or location; pseudonymous learner ID	Enforced by schema
N9	Decision parity between Python and TypeScript engines	100% on shared vectors

Mapping to hackathon criteria

Criterion	Where it is satisfied
Problem understanding	Section 2
Technicalities	Sections 5 to 9, 13, 15
Pitch and presentation	Section 17
Bonus: Odia	Sections 11, 12
Bonus: ease of use	F1, F4, Section 14 UX rules
Bonus: offline	Sections 9, 10
Bonus: agentic framework	Section 4

4. Agentic approach

Decision (Decided): a deterministic workflow agent built as a LangGraph state machine, with bounded autonomy inside two nodes. The graph chooses the learning action by explicit rules over learner state and environment state. The LLM generates explanations and practice questions inside nodes and never chooses the action.

Why deterministic at the core

- The next action must be explainable to a judge, a teacher and a parent. Rules over mastery and connectivity are auditable; a free-running LLM planner is not.
- The same decision must run on the phone with no network. A rule engine fits in a few kilobytes of JavaScript; an autonomous planner needs a hosted model.

- Decisions must be testable against independently labelled expected actions (Section 15).
- LLM output in Odia is uneven in quality. Keeping the model out of control flow limits a bad generation to one answer, not the learner's path.

Where autonomy is allowed (bounded)

Node	Autonomy	Bound
Doubt answering	A small tool-using loop chooses among retrieve, rephrase query, glossary lookup, then answers	Max 3 tool calls, whitelisted tools, 12 s budget, one retry per tool, then the extractive fallback
Question drafting	LLM drafts a practice question from a retrieved chunk	Must pass schema and answer-key validation, else a curated bank question is used

What breaks if we choose the other way

If we chose fully autonomous	If we chose a pure fixed workflow
Offline mode fails because the planner needs a cloud model	Doubt answering cannot adapt to unclear or multi-part questions
Decisions differ between runs, so tests and audits fail	Every doubt takes the same retrieval path even when it clearly fails
A wrong plan can send the learner down a bad path with no guard	Easy to build and test, so we keep it for everything else

What is reproducible and what is auditable (Decided). These are different guarantees and must not be confused.

Part	Guarantee	How
Learning action, target concept, delivery mode, background actions, reason code	Reproducible: the same normalised state, environment snapshot and ruleset version always give the same result	Pure functions, integer mastery, shared test vectors (Section 15)

Part	Guarantee	How
Generated Odia text	Auditable and replayable from the record, not identical on rerun	A generation record stores provider, model ID, prompt version and hash, retrieved chunk IDs, parameters, output, validation results and any fallback used (Section 7). Identical text on rerun is not promised unless verified with the actual provider and settings

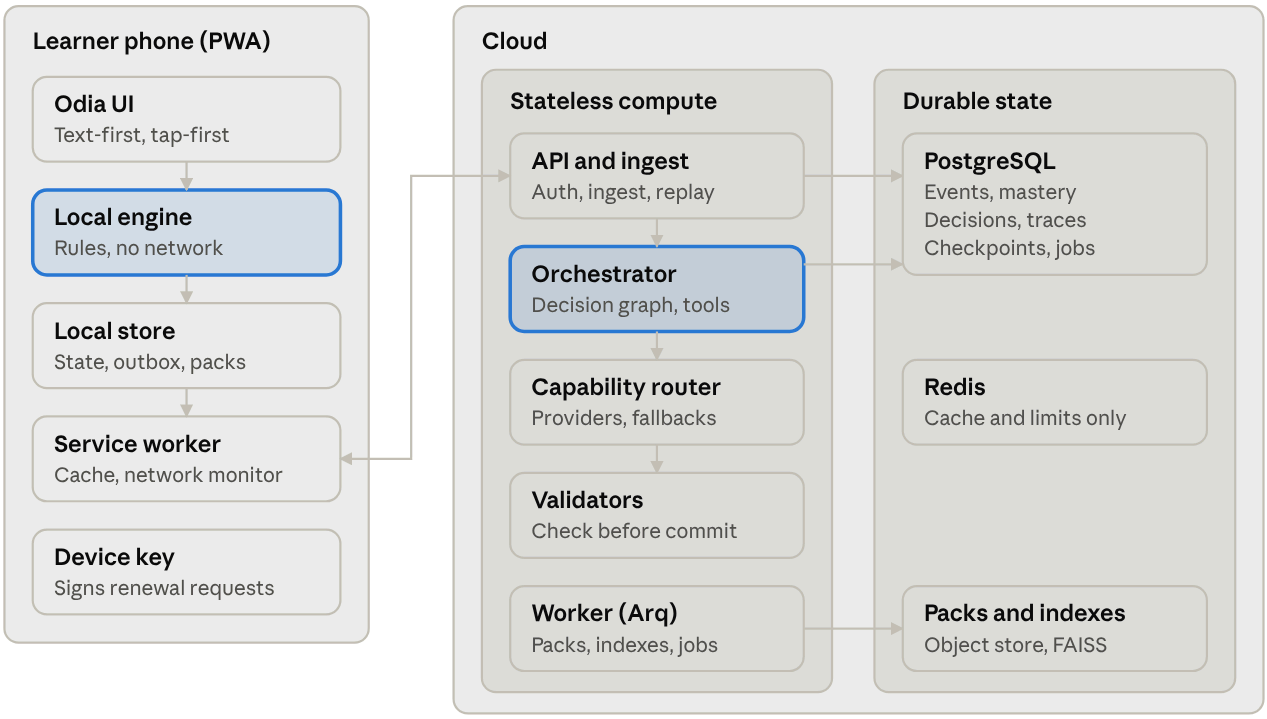
Framework (Decided): LangGraph on the server for orchestration, with checkpoints in PostgreSQL. A TypeScript implementation of the decision subgraph (pure functions, no LLM, no network) runs on the client. Both load the same versioned `rules.json` and must pass the same test vectors, so the two cannot silently disagree.

LLM provider policy (Decided): the generator sits behind the provider interface of Section 5. Candidates are evaluated by measurement (Section 12). No provider is placed in the core decision loop, and the local rule fallback always remains.

5. System architecture

The phone can make every learning decision alone. The cloud adds heavier AI and the reconciled record, using stateless processes over durable stores.

Compute is stateless; learner state lives in durable stores



Compute is stateless and restartable; learner state lives only in the durable layer.
The phone owns pending events until sync acknowledges them; Redis is cache only.

system architecture · stateless compute, durable state, phone

Statelessness boundary (Decided). Only the compute layer is stateless: API, orchestrator, validators and workers keep nothing in process memory that correctness depends on, so any of them can be killed and restarted at any time. The system as a whole intentionally persists learner state, in the durable layer.

State and write ownership (Decided)

Data	Single writer	Durable home	Never written by
Pending learner events while offline	Phone (local store, one transaction with the state change)	IndexedDB outbox	Server
Accepted events, mastery projection, sync cursors	API ingest service, in one transaction	PostgreSQL	Orchestrator, AI providers, client

Data	Single writer	Durable home	Never written by
Decisions, traces, graph checkpoints, generation records	Orchestrator	PostgreSQL	AI providers, validators
Content metadata, indexes, packs, manifests	Worker, only through the publication gate	PostgreSQL, object store, index files	API request handlers
Job state	Worker and the job reconciler	PostgreSQL job table	Redis alone
Rate limits and short caches	API	Redis	Anything that needs them for correctness

The phone owns events until the server acknowledges them. After sync, the server owns the reconciled, authoritative state. AI providers and generators never write to a database; they return typed results to the router.

Transaction boundary for ingestion (Decided). One database transaction inserts the batch of events (idempotent on device_id and seq), applies accepted events to the mastery projection in canonical order, and advances the device cursor. The server decision is computed after that commit, so a decision failure never rolls back accepted events. If the decision cannot be computed, the response says so and the phone keeps its local decision.

Component responsibilities

Component	Owns	Must never do
Odia UI	Rendering, input, audio playback	Hold business rules
Local engine	Next action from local state and network state, with trace	Call the network or an LLM
Local store	Profile-scoped mastery, outbox, quarantine, packs, decisions	Write an event without its state change
Service worker	App shell and pack caching, network probe	Decide learning actions

Component	Owns	Must never do
API and ingest	Auth, validation, rate limits, idempotency, event ingestion and projection	Contain orchestration logic
Orchestrator	The decision graph, tool execution, checkpoints	Let the LLM choose the action
Capability router	Calls providers, applies timeouts, retries and fallbacks	Write to databases
Validators	Ordered checks of Section 13	Alter learner state
Worker	Pack building, embeddings, index builds, queued jobs	Publish anything that fails the gate

Capability router and fallback matrix (Decided). The rest of the system depends on typed interfaces, never on a provider. Cloud calls need connectivity; offline, only local rules, cached data and models actually shipped on the device are available. No on-device LLM is required: offline doubts are answered extractively from the installed pack.

Capability	Primary	Degraded	Final fallback	Offline
Retrieval	Hybrid dense plus keyword on the server	Keyword-only server index	Local keyword (BM25) over installed pack	Local only
Generation	Primary LLM provider	Second provider behind the same interface	Extractive answer from the best chunk, labelled basic	Extractive only
ASR	Hosted Odia ASR	Second candidate if evaluated	Typed input; never guess silently	Not available
TTS	Hosted Odia TTS	Pre-generated pack audio	Text only	Pack audio if installed

Capability	Primary	Degraded	Final fallback	Offline
Validation	Model entailment check plus deterministic checks	Deterministic checks only, which forces an extractive answer or a redirect	Redirect or refuse	Deterministic checks only
Storage and sync	Transactional ingestion	Retry with backoff; events stay in outbox	Learning continues locally	Outbox only
Pack download	Resumable range download	Retry from last verified part	Keep the old pack	Installed packs
Configuration	Remote config with TTL cache	Last known good config	Ruleset bundled in the app and pack	Bundled ruleset
Analytics	Counters uploaded with sync	Local counters only	Dropped silently; never blocks learning	Local counters

Provider contract (Decided). Every provider implementation returns the same typed result.

```
class ProviderError(BaseModel):
    code: Literal["TIMEOUT", "RATE_LIMITED", "UNAVAILABLE", "BAD_OUTPUT",
                  "LOW_CONFIDENCE", "UNSUPPORTED_LANGUAGE",
"QUOTA_EXHAUSTED"]
    retryable: bool
    detail_dev: str                      # never shown to learners

class Ok(BaseModel, Generic[T]):
    value: T
    provider: str
    model_id: str
    latency_ms: int
    request_id: str

Result = Ok[T] | ProviderError

class Generator(Protocol):
    async def generate(self, req: GenerateRequest, *, timeout_s: float) ->
Result[GenerateResponse]: ...
# Retriever, Transcriber, Synthesizer and Entailer follow the same shape.
```

Storage roles (Decided)

Store	Role	Rules
PostgreSQL 16	System of record: learners, devices, profiles, events, mastery projection, decisions, traces, generation records, content metadata, pack manifests, jobs	Alembic migrations, tested against an empty database and the previous release snapshot in CI; applied migrations are never edited
Object storage	Source assets and downloadable pack artifacts	Packs written under a temporary key, verified, then promoted atomically
FAISS index files	Server retrieval index	Versioned per content release; built by the worker; loaded read-only by the API; an active-index pointer row in PostgreSQL switches versions; rebuild on every content publish

Store	Role	Rules
Device keyword index	Offline retrieval	Compact index built at pack install time
Redis	Cache and rate limits only	Never the only copy of learner state or pending work; losing it loses nothing durable

Backups and restore (Decided). Nightly logical backup of PostgreSQL and a restore drill before the checkpoint and before the demo: restore into a scratch database, then verify that mastery rebuilt by replaying the restored events equals the restored projection. A backup that has not been restored is not counted as a backup.

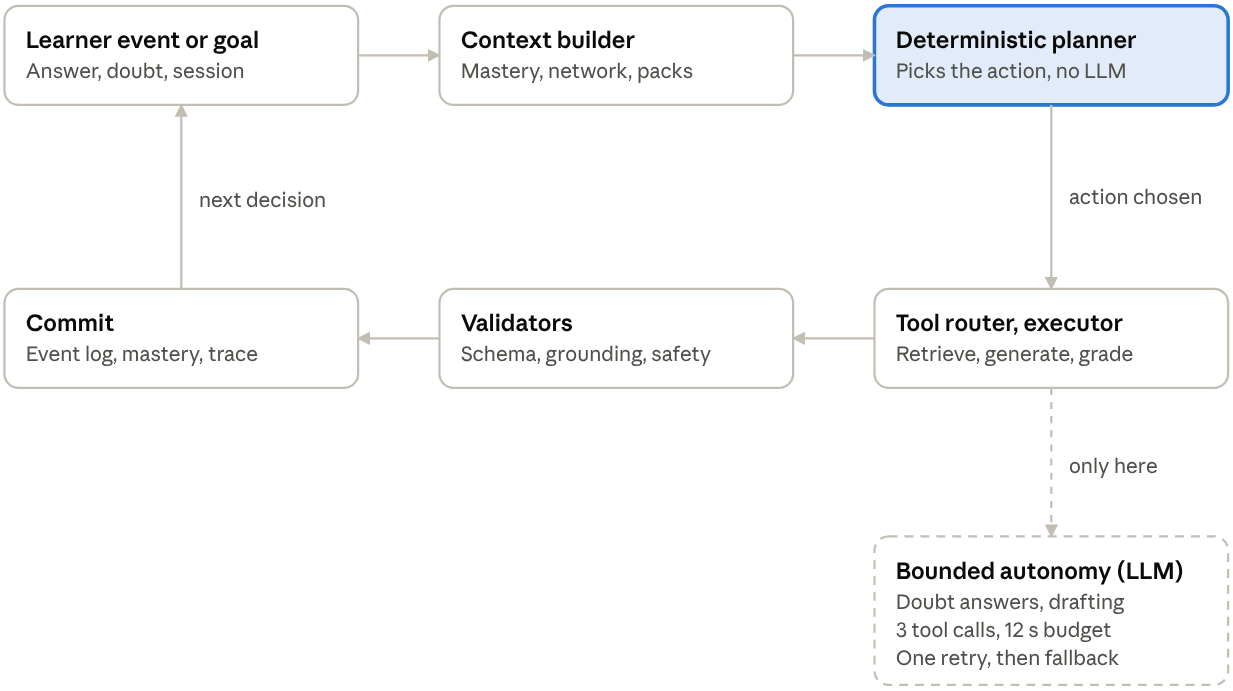
Worker model (Decided). Arq runs the jobs; the PostgreSQL job table is the source of truth, so a lost queue loses nothing.

Aspect	Rule
Job record	job_id, type, payload, dedupe_key, status (queued, running, succeeded, retry, dead), attempts, max_attempts, next_run_at, lease_expires_at, last_error
Recovery	A reconciler runs every minute and re-enqueues any job that is queued or running with an expired lease
Retryable errors	Timeouts, rate limits, 5xx: exponential backoff from 5 s, factor 2, plus or minus 20% jitter, cap 10 min, at most 6 attempts
Non-retryable errors	Failed publication gate, invalid input: go straight to dead
Idempotency	Jobs carry a dedupe key such as pack:{pack_id}:{version}; running a job twice produces the same artifact
Visibility	Dead jobs are listed by an admin endpoint and counted in the developer panel

6. Agent design

Every learner event runs through one loop that ends in a committed decision. The planner is rule-based; the language model is called only by the tool executor, only for the two bounded tasks of Section 4.

The planner picks the action; the model works only inside bounded tools



Every step appends a trace entry; a failure ends in a typed fallback, never an open loop.

agent loop · planner, tools, validators, commit

Loop stages (Decided)

Stage	What it does	Runs as	On failure
Learner event or goal	A graded answer, a doubt, a session start, or a connectivity change arrives	Input	Invalid payload is rejected with a typed error

Stage	What it does	Runs as	On failure
Context builder	Loads profile-scoped mastery, last 20 events, environment snapshot, installed packs; normalises them	Deterministic code	Missing or corrupt state gives reason STATE_INVALID_FALLBACK
Deterministic planner	Chooses learning action, target concept, delivery mode, background actions, reason code	Pure function over rules.json	Unknown ruleset version gives a typed error and the fallback decision
Tool router and executor	Runs retrieval, generation and grading tools within limits	Typed tools; LLM only inside the bounded tasks	Retry once, then provider fallback, then extractive answer
Validators	Ordered checks of Section 13 before anything is shown or stored	Mostly deterministic	Safe default decision or extractive answer
Commit	Writes the event, mastery projection, decision and trace	Single writer per Section 5	Retry; events remain in the outbox

After commit the loop waits for the next event. The next decision always starts from committed state, never from a prior run's memory.

Decision contract (Decided). Every decision returned by the server or the local engine contains exactly one learning action, a feasible delivery mode, zero or more background actions, a stable reason code and the ruleset version.

Part	Values	Meaning
Learning action	REMEDIATE, RETRY, REVISE, ADVANCE	What the learner does next
Delivery mode	cached, rich, prefetch_then_offline	How content reaches the device. Offline always means cached
Background actions	SYNC, PREFETCH (sorted, may be empty)	Silent work that never interrupts learning
Reason code	Enum in Appendix A	Stable, machine-readable explanation
needs_help	true or false	Set only when all interventions are exhausted

Before a decision is saved or shown, it must pass the trajectory check: the target concept exists, the action and reason code are in their enums, the delivery mode is feasible for the installed packs and network state, and the ruleset version is known. A failing decision is replaced by the fallback decision (reason STATE_INVALID_FALLBACK) and logged at high severity.

Normalised input and mastery representation (Decided). Mastery is stored and compared as integer basis points (0 to 10000), so Python and TypeScript cannot disagree through floating-point rounding. Concept lists are sorted by concept ID, timestamps are not inputs to the planner, and the environment snapshot is reduced to the network state, free-storage bucket and installed pack versions.

Which events change mastery (Decided). Only graded answers update mastery. Viewing content or asking a doubt is never treated as proof of learning.

Event	Updates mastery	Other effect
ANSWER_SUBMITTED, valid, first for its attempt_id	Yes, using the answer key of the question revision it was made against; the server regrades from its own key and its result wins	Mismatch with the client's claimed result is counted

Event	Updates mastery	Other effect
ANSWER_SUBMITTED, repeated attempt_id	No	Accepted with effect NONE, reason REPEAT_SUBMISSION
ANSWER_SUBMITTED, invalid payload	No	Rejected, reason INVALID_PAYLOAD
ANSWER_SUBMITTED, question revision unknown or marked legacy	No	Accepted with effect NONE, reason LEGACY_QUESTION
QUESTION_ABANDONED (shown, left without answering)	No	Counted as an unanswered attempt for reporting, not toward attempts without gain
CONTENT_VIEWED	No	Resets the view counter used by the REVISE rule
DOUBT_ASKED, DOUBT_ANSWERED	No	Sets a concept hint for target selection
SESSION_STARTED, SESSION_ENDED, PACK_INSTALLED, NETWORK_STATE_CHANGED, DECISION_MADE_LOCAL	No	Stored for context and audit

Bayesian Knowledge Tracing update (Decided). For a graded answer on a concept with stored mastery p (basis points divided by 10000, or p_init if the concept is new): if correct, posterior = p(1 - slip) / (p(1 - slip) + (1 - p) guess); if wrong, posterior = p slip / (p slip + (1 - p)(1 - guess)). Then p_next = posterior + (1 - posterior) learn, clamped to 0.01 to 0.99, and stored as basis points rounded half up. Each update starts from the stored integer, so replay is exact. Parameters live in rules.json under a `bkt_version`; changing them bumps the version and forces a full projection rebuild by replay.

Learning-action rules (Decided). T is the target concept. Thresholds are in basis points in rules.json. The first matching rule wins.

Order	Condition	Action and reason code
0	State invalid, ruleset unknown	Fallback decision: RETRY on the last concept, or ADVANCE to the first concept if none. Reason STATE_INVALID_FALLBACK
1	Learner has no mastery rows	ADVANCE to the first concept with no prerequisites. Reason COLD_START
2	Guard: 3 or more graded answers on T without a gain of at least 200 bp	See guard steps below
3	p below 4000 and a prerequisite below 4000	REMEDIATE the weakest prerequisite. Reason WEAK_PREREQ_GAP
4	p below 4000	REMEDIATE T with simpler content. Reason WEAK_CONCEPT
5	p 4000 to 8499 and last graded answer wrong	RETRY at the same or easier difficulty. Reason WRONG_RETRY
6	p 4000 to 8499 and last graded answer correct	RETRY at the same or next difficulty. Reason PARTIAL_PRACTICE
7	p 8500 or above	ADVANCE to the next eligible concept (below). Reason MASTERED_ADVANCE

Guard steps (Decided). ADVANCE always means moving to a new concept; the guard never uses it to mean a different content type. When repeated attempts show no improvement:

1. If no alternate content format has been shown for T in this cycle: REVISE with an alternate format such as a worked example. Reason NO_GAIN_ALT_CONTENT.
2. Otherwise, if a prerequisite of T is below mastered: REMEDIATE the weakest one. Reason NO_GAIN_REMEDIATE_PREREQ.
3. Otherwise this is the terminal state: REVISE T with needs_help set. Reason INTERVENTIONS_EXHAUSTED. The learner is never advanced while T is below mastered, the Progress screen shows an Odia prompt to ask a teacher, and the case appears in the developer panel.

The guard cycle resets when T's mastery rises by at least 200 bp.

Terminal and edge behaviours (Decided)

Situation	Behaviour
No eligible next concept because every concept in the installed slice is mastered	REVISE the mastered concept with the oldest last-seen time. Reason SLICE_COMPLETE; UI shows a completion message
No eligible next concept because prerequisites are not mastered	REMEDIATE the weakest unmet prerequisite. Reason BLOCKED_BY_PREREQ
Required pack missing and the network is offline	Use the nearest installed concept by graph distance, ties by chapter order. Reason CONTENT_NOT_AVAILABLE_OFFLINE. PREFETCH is added for when the network returns
No pack installed at all	Cannot occur in normal use because the app ships with a seed pack; if storage corruption removes it, the fallback decision is returned and the UI asks the learner to reconnect and download

Target concept and next eligible concept. The target is the concept of the last graded answer or, failing that, the doubt's concept hint; otherwise the concept with the lowest mastery among those whose prerequisites are all mastered, ties by chapter order. The next eligible concept is the first concept in chapter order that is not mastered and whose prerequisites are all at or above 8500.

Delivery and background rules. Delivery follows the network state of Section 10: offline and weak give cached, online gives rich when the server has newer content, and prefetch_then_offline when the target's pack is missing and the network is good. SYNC is added whenever the outbox is not empty and the network is not offline. PREFETCH follows the rule in Section 10.

Decision trace (Decided). Every stage appends a step with name, inputs, output and duration, so a judge can see which tools ran, in what order and why. Traces are stored with the decision and shown in the developer panel.

```
{
  "decision_id": "uuid",
  "ruleset_version": "1.0.0",
  "steps": [
    { "stage": "context", "ms": 3, "out": { "concept": "uuid", "p_bp": 3100 }
},
    { "stage": "planner", "ms": 1, "out": { "rule": 4, "action": "REMEDIATE",
"reason": "WEAK_CONCEPT" } },
    { "stage": "delivery", "ms": 1, "out": { "delivery": "cached",
"background": [] } },
    { "stage": "trajectory_check", "ms": 1, "out": { "ok": true } }
  ]
}
```

7. Data model

PostgreSQL is the system of record; the phone holds a profile-scoped subset in IndexedDB. Learner IDs are random UUIDs. No phone number, location or real name is stored (N8). All timestamps are UTC.

Device and profile are separate (Decided, fixes the v1 model). A physical phone is one device record. Each learner who uses that phone is a profile linked to it through `device_profiles`. In the MVP a learner has exactly one active device (Section 9), enforced by a partial unique index on active links.

PostgreSQL tables (Decided)

Table	Key columns	Purpose
learners	id uuid, display_name, language, created_at	Pseudonymous learner
devices	id uuid, public_key, app_version, created_at, last_seen_at, revoked_at	One row per physical phone; public key verifies renewal signatures
device_profiles	device_id, learner_id, status (active, unlinked), linked_at	Which learners use which phone; unique active link per learner
chapters, concepts	stable slug id, titles in Odia and English, prerequisite ids	Curriculum and knowledge graph

Table	Key columns	Purpose
content_chunks	id, concept_id, text_or, glossary terms, content_version	Retrieval units of about 80 to 200 Odia words
questions	question_id (stable slug), concept_id	Stable identity of a question across versions
question_revisions	question_id, revision, stem_or, options, answer_key, answer_key_hash, difficulty, misconception_tag, status (active, legacy)	Each wording or key change is a new revision; events record the revision they were made against
mastery	learner_id, concept_id, p_bp int, attempts, correct, last_seen_at, last_event_seq, bkt_version	Projection rebuilt by replay; never a source of truth
events	event_id, device_id, seq, learner_id, type, payload, client_ts, server_ts, status (accepted, rejected), effect (mastery_updated, none), reason	Append-only; unique on (device_id, seq)
decisions	id, learner_id, trigger_event_id, action, target_concept_id, delivery, background text[], reason_code, needs_help, inputs jsonb, trace jsonb, ruleset_version, source (server, local), created_at	Every decision with full trace
generation_records	id, decision_id, provider, model_id, prompt_version, prompt_hash, params, chunk_ids uuid[], output_or, validation_results jsonb, fallback_used, created_at	Audit record for every generated text
packs	id, version, chapter_ids, size_bytes, sha256, min_app_version, status (draft, published)	Downloadable learning packs

Table	Key columns	Purpose
jobs	job_id, type, payload, dedupe_key, status, attempts, max_attempts, next_run_at, lease_expires_at, last_error	Durable worker jobs (Section 5)
sync_cursors	device_id, last_acked_seq, last_pack_version	Per-device position

The LangGraph checkpointer adds its own tables. Checkpoints support audit and resume of long doubt runs; correctness never depends on them, because every decision is recomputable from committed state.

Stable identifiers (Decided). Concept and question IDs are human-readable slugs (for example `sci9.ch1.c03.q07`) that never change. A change to a question's wording or answer key creates a new revision of the same question ID. This lets an old pack keep working while a learner is offline, and lets the server interpret old events correctly after a pack upgrade.

Local store layout (Decided). Two IndexedDB databases per phone, accessed only through one `ProfileSession` module so no code path can read another profile's data.

Database	Stores	Isolation rule
saarthi-shared	device meta, device key handle, installed packs, chunks, questions, keyword index	Content only; contains nothing personal
saarthi-profile-{learner_id}, one per profile	profile and PIN hash, mastery, outbox, quarantine, decisions, session and UI cache, sync cursor	Opened only after profile selection; closed and all in-memory state discarded on switch

The PIN separates profiles on a shared phone at the interface level. It is not server authentication and it is not encryption at rest; the MVP does not claim protection against someone with full access to the phone's storage. Service worker caches never store API responses that contain learner data.

Mastery model. Bayesian Knowledge Tracing per concept with the update rule and integer basis-point storage of Section 6. Thresholds and parameters live in rules.json (Appendix A).

Learning pack format (Decided)

```
{
  "pack_id": "sci9-ch1",
  "version": 3,
  "language": "or",
  "chapters": ["sci9.ch1"],
  "files": {
    "chunks": "chunks.jsonl",
    "questions": "questions.json",
    "glossary": "glossary.json",
    "concept_graph": "graph.json",
    "audio": "audio/"
  },
  "ruleset_version": "1.0.0",
  "size_bytes": 4200000,
  "sha256": "<checksum>",
  "min_app_version": "1.0.0"
}
```

Packs are text-first. Pre-generated audio is optional and downloaded only on a good connection.

8. API, sync outcomes and authentication

REST over HTTPS with JSON bodies, FastAPI with Pydantic models as the contract. Base path `/api/v1`. The OpenAPI file is generated from the models and committed; the TypeScript client is generated from it in CI.

Conventions (Decided)

- Errors use RFC 7807 problem+json with a stable `code` field. Every response carries `request_id`; every decision carries `decision_id`.
- Every POST other than `/sync` accepts an `Idempotency-Key` header and replays the stored response for 24 hours. `/sync` is idempotent through event keys.
- Learner-facing text fields are suffixed `_or` (Odia); other text is for developers and is never shown to learners.

Endpoints

Method and path	Purpose	Offline equivalent on the phone
POST /devices	Register a device with its public key; returns tokens	Deferred until first connection
POST /learners	Create a learner and link it to the calling device	Profile created locally, registered at first sync
POST /auth/challenge, POST /auth/renew	Renew tokens by refresh token or by signed challenge	None needed to keep learning
POST /sync	Upload outbox events, receive per-event outcomes, mastery, a decision and pack updates	Outbox queue
POST /decide	Compute the next action for a learner	Local engine
POST /doubts	Answer a text doubt with grounded retrieval	Extractive answer from the installed pack
POST /asr, POST /tts	Online Odia speech to text and text to speech (P1)	Not available; text only
GET /questions/next	Next question for a concept and difficulty	Local question bank
GET /packs/manifest, GET /packs/{id}	Pack list and resumable download (HTTP Range)	Installed packs
GET /config	Remote configuration with TTL	Bundled configuration
GET /decisions/{id}/trace	Full trace	Local decisions store
GET /health, GET /ready	Liveness and readiness	None
GET /admin/jobs	Dead and retrying jobs (operators only)	None

Event types (Decided): SESSION_STARTED, SESSION_ENDED, CONTENT_VIEWED, ANSWER_SUBMITTED, QUESTION_ABANDONED, DOUBT_ASKED, DOUBT_ANSWERED, PACK_INSTALLED, DECISION_MADE_LOCAL, NETWORK_STATE_CHANGED.

Core schemas (Pydantic is the source; shown as JSON)

```
// Event: created on the phone, immutable
{
  "event_id": "uuid",
  "device_id": "uuid",
  "learner_id": "uuid",
  "seq": 1042,
  "type": "ANSWER_SUBMITTED",
  "client_ts": "2026-10-11T16:02:11Z",
  "payload": { "attempt_id": "uuid", "question_id": "sci9.ch1.c03.q07",
"revision": 2,
               "response": "B", "correct_claim": false, "time_ms": 18400 }
}

// Environment snapshot
{ "network": "online | weak | offline", "rtt_ms": 840, "downlink_kbps": 120,
  "storage_free_mb": 310, "installed_pack_versions": { "sci9-ch1": 3 },
"battery_saver": false }

// Decision: server or local engine
{ "decision_id": "uuid", "action": "REMEDIATE", "target_concept_id":
"sci9.ch1.c03",
  "delivery": "cached", "background": ["SYNC"], "reason_code":
"WEAK_CONCEPT",
  "needs_help": false, "reason_or": "<one-line Odia reason>",
  "ruleset_version": "1.0.0", "source": "server" }
```

Per-event sync outcomes (Decided). `POST /sync` carries up to 100 events of one learner in ascending seq order and returns one result per event, not a sequence range.

```
{ "results": [ { "event_id": "uuid", "seq": 1042, "outcome": "ACCEPTED",
                 "effect": "mastery_updated", "reason": null } ],
  "mastery": [ ... ], "decision": { ... },
  "decision_status": "computed | deferred", "pack_updates": [ ... ],
"server_cursor": 1042 }
```

Outcome	Meaning	Examples of reason	Phone action
ACCEPTED	Stored; effect is mastery_updated or none	REPEAT_SUBMISSION, LEGACY_QUESTION (effect none)	Remove from outbox
DUPLICATE	Same event_id and payload already stored earlier	None	Remove from outbox
RETRYABLE	Not stored; may succeed later	SERVER_BUSY, DB_TIMEOUT	Keep in outbox, back off
REJECTED	Permanently invalid; stored as a rejected tombstone so its seq is consumed	INVALID_PAYLOAD, UNKNOWN_EVENT_TYPE, LEARNER_NOT_LINKED, DEVICE_REVOKED, PAYLOAD_CONFLICT	Move to quarantine, then remove from outbox in one transaction

- An event leaves the outbox only on ACCEPTED, DUPLICATE or a completed move to quarantine. An event missing from the response (for example a lost response) stays and is resent; resending is safe because of the (device_id, seq) uniqueness constraint and the event_id key.
- A rejected event never blocks later events: it has no mastery effect and later events proceed. Quarantined events are kept on the phone and visible in the developer panel; they are never silently dropped.
- PAYLOAD_CONFLICT means the same (device_id, seq) arrived with a different event_id or payload. It indicates a client bug or tampering, is rejected, and is logged at high severity.
- The batch is one database transaction. If it fails, every event in it is returned RETRYABLE with HTTP 503 and none is ACCEPTED. The server decision is computed after commit; if it fails, `decision_status` is `deferred` and the phone keeps its local decision.
- If a server decision arrives while the learner is mid-activity, the phone stores it as a pending suggestion and shows it only at the next natural break (Section 10).

Authentication and recovery after long offline periods (Decided). Authentication protects network calls only; it never gates local learning.

1. Registration: the phone generates a non-extractable ECDSA P-256 key pair with WebCrypto and registers the public key at `POST /devices`. It receives a short-lived access token (24 hours) and a rotating refresh token (30 days). The token identifies the device; each event is authorised by the active device_profiles link of its learner.
2. Normal renewal: on reconnect, before syncing, the phone renews with the refresh token.
3. Long offline period: if the refresh token has also expired, the phone calls `/auth/challenge`, signs the returned nonce with its device key and calls `/auth/renew`. No password or PIN is involved, and the outbox is untouched.
4. While auth is unavailable the app shows no error; sync is paused in an AUTH_PENDING state, installed packs and local decisions keep working, and pending events stay queued.
5. If the server answers DEVICE_REVOKED, pending events are moved to quarantine, not deleted, and the learner can keep learning locally.
6. If the device key is lost (site data cleared), the local database and outbox are lost with it, so nothing pending can be stranded; the phone registers as a new device. Re-linking a learner to a new device is out of scope for the MVP and is recorded as a limitation.
7. The local profile PIN is never sent to the server and is never treated as authentication.

Error codes

Code	HTTP	Client behaviour
AUTH_EXPIRED	401	Renew as above, retry once, otherwise AUTH_PENDING
DEVICE_REVOKED	403	Quarantine pending events, continue locally
VALIDATION_FAILED	422	Do not retry; log; generic Odia message
CONTENT_VERSION_STALE	409	Fetch manifest, update pack, replan
LLM_UNAVAILABLE	503	Extractive fallback, answer marked basic
RATE_LIMITED	429	Back off with jitter; keep in outbox
UPSTREAM_TIMEOUT	504	Retry with backoff up to 5 times, then queue

9. Offline-first client, event ordering and replay

Principle (Decided). The phone is authoritative for what the learner just did; the server is authoritative for the reconciled state. Learner actions are appended as immutable events and mastery is derived from them, so sync merges event logs and never has to merge conflicting values.

Client architecture

- React PWA built with Vite and TypeScript, installable, with a Workbox service worker.
- App shell and installed packs are cache-first. API calls are network-first with a 6 s timeout, then fall back to local logic.
- All state changes go through the `ProfileSession` store module, which writes the state change and its outbox event in a single IndexedDB transaction, so one can never exist without the other.
- A `NetworkMonitor` produces the network state of Section 10.
- Storage guard: request persistent storage at first run, warn at 50 MB free, never evict an installed pack without the learner's consent.

Canonical event ordering (Decided). Bayesian Knowledge Tracing is sequential, so the order of answer events changes the result and must be fully defined.

Question	Rule
What orders a learner's events?	Ascending seq on the learner's one active device. Seq comes from a device-level monotonic counter that is incremented and persisted in the same transaction as the event
Are timestamps used for ordering?	No. client_ts is for display and analytics only; server_ts is for audit. Clock skew therefore cannot change mastery
What if events arrive late, twice or out of order?	Duplicates return DUPLICATE. For a late event, the server inserts it and rebuilds that learner's projection by replaying all accepted events in seq order. Incremental update is used only when every new seq is above the last applied seq
What about rejected events?	They are tombstones with no effect and are skipped in replay

Question	Rule
Are cross-device causal guarantees offered?	No. MVP correctness is scoped to one active device per learner. Several learners may share one device. A second device for the same learner is rejected with LEARNER_NOT_LINKED

Multi-device learners are deferred until a merge policy (per-device logs plus a deterministic causal merge) is written, implemented and tested; until then the limitation is stated plainly.

Replay invariant (Decided). Replaying the same accepted event log in canonical order always yields the same mastery projection and, from it, the same decision. This is tested by property tests in Section 15 and is the oracle for the fault matrix.

Sync protocol (Decided)

1. Trigger: the network becomes online or usable-weak, the app returns to the foreground, or 5 minutes pass during an online session. Before syncing, auth is renewed if needed (Section 8).
2. For one profile at a time, the phone sends its oldest 100 outbox events in seq order with the environment snapshot to `POST /sync`.
3. The server validates and, in one transaction, inserts the events, applies accepted ones to the projection in canonical order and advances the cursor. It returns a result per event.
4. After commit the server computes a decision and returns it with authoritative mastery and any pack updates.
5. The phone applies the outcome rules of Section 8, replaces local mastery with the server's, then re-applies its own events created after the upload, in seq order, with the same update function (rebase).
6. If outbox events remain, repeat from step 2. A partial failure leaves unacknowledged events untouched.
7. If the server decision conflicts with what the learner is currently doing, it is stored as a pending suggestion and shown at the next natural break (Section 10).

Conflict and fault rules

Situation	Rule
Same event uploaded twice	DUPLICATE; removed from outbox
Same (device_id, seq) with a different payload	REJECTED as PAYLOAD_CONFLICT; quarantined
Device clock wrong	No effect on ordering; flagged for diagnosis only
Pack version changed while offline	Old pack keeps working. Events keep the question revision they were made against; unknown or legacy revisions are accepted with no mastery effect
Ruleset version differs between phone and server	The server decision wins on sync; the phone's earlier decisions stay in the trace as source local
Learner switches profile on a shared phone	Separate database, outbox and cursor per profile; sync uploads each in turn
Auth expired	Sync paused in AUTH_PENDING; local learning continues (Section 8)

Failure behaviour. Sync never blocks learning. A failed sync records a SYNC_FAILED trace entry and retries with exponential backoff (5 s, 15 s, 60 s, 300 s) with jitter. Learners see no error. A permanently rejected event is quarantined and counted in the developer panel.

Acceptance test. A fixed set of at least 200 events across 10 offline cycles, with random app kills and injected request failures, must reach the server exactly once, and the server's mastery must equal a canonical replay of the accepted log (N5, Section 15).

10. Connectivity-aware planning

Connectivity is an input to the decision, not an on and off switch. The phone classifies the network by measuring it, not by trusting `navigator.onLine`, which reports a phone with a signal but no working data as online.

Network classification (Default, tune during weak-network testing)

State	Condition	Measured by
online	Probe succeeds, RTT under 800 ms, downlink at least 400 kbps	Active probe plus a rolling download sample
weak	Probe succeeds but RTT 800 to 3000 ms or downlink 50 to 400 kbps	Same
offline	Two consecutive probe failures, or `navigator.onLine` is false	Same

- Probe: a HEAD request to `/health` every 30 s while the app is visible, 3 s timeout. Network Information API values are hints only.
- Hysteresis: downgrade after one bad probe; upgrade only after two good ones, so the state does not flap.
- Every change is logged as NETWORK_STATE_CHANGED and enters each decision as the network state.

What each state changes (Decided)

Aspect	Online	Weak	Offline	Reconnected
Content	Fresh or rich content from the server	Cached, text only, no audio or images	Installed pack only	Unchanged until a natural break
Doubt answer	Server retrieval and generated Odia answer	Server answer with a short cap and 6 s timeout, else local extractive	Local extractive answer from the pack, labelled basic	Unchanged until sync finishes
Voice	Online ASR and TTS if enabled (P1)	Text only	Text only; pack audio if installed	Text only until online state is stable
Assessment	Same question bank, local grading	Local	Local	Local

Aspect	Online	Weak	Offline	Reconnected
Decision	Local engine; server decision on sync	Local engine	Local engine	Server decision stored as a pending suggestion
Background work	Sync, pack updates, prefetch	Small sync batches, resumable pack download in parts	None; events queue	Sync first, then pack updates, then replan

The differences are in what is delivered and which tools may run; the learning action rules of Section 6 are the same in every state.

Non-blocking reconnect (Decided). Reconnecting never interrupts the learner. Sync runs in the background. If the server returns a decision that differs from the local one, it is stored as a pending suggestion. The next-action card on the Home screen updates at the next natural break: after the current question is answered, when a lesson ends, or when the learner returns to Home. The UI never replaces content the learner is reading or a question they are answering.

Prefetch rule (Decided). When the network is online, free storage is at least 150 MB, battery saver is off, and the learner's target concept or its graph neighbours are not installed, the planner adds PREFETCH. If the learner's recent history shows a large offline share, the planner prefers a target that is already installed, so the next session can be completed offline.

Download behaviour. HTTP Range requests in 512 KB parts, resumable after any interruption, checksum verified before install, and the old pack kept until the new one verifies.

11. Content and RAG pipeline

Content quality decides whether learners and teachers trust the product, so content is a managed pipeline with human review and a publication gate. The MVP covers three Class 9 Science chapters end to end.

End-to-end flow (Decided)

1. Source material and rights check. Confirm permission to use and redistribute each source before it enters the pipeline (Section 18, D1). No pack is published without it.
2. Extraction. Copy text from digital sources, or OCR scanned pages and correct by hand.
3. Odia normalisation. Unicode NFC, layout artefacts removed, digit forms recorded.
4. Concept and prerequisite tagging. A subject expert and the Odia lead split each chapter into concepts with stable slug IDs and prerequisite links.
5. Chunking and glossary. 80 to 200 Odia words per chunk, one idea each, with concept ID, textbook page reference and glossary terms. The glossary maps technical terms between Odia and English.
6. Question bank. At least 12 questions per concept (4 per difficulty), each with an answer key and a misconception tag on every wrong option. LLM drafts are allowed, but every item is reviewed by a human before entering the bank.
7. Indexes. Dense embeddings and a keyword index for the server; a compact keyword index built on the phone at install time.
8. Pack build. Write the pack files of Section 7, compute checksums, store under a temporary key.
9. Publication gate. The pack is promoted and listed in the manifest only if every check below passes.
10. Device installation. The phone downloads in resumable parts, verifies the checksum, installs, builds its local index, and records PACK_INSTALLED.

Publication gate (Decided). A pack is not published, and the job goes to dead with a visible reason, if any of these fail:

- Every required Odia field is present and non-empty.
- Every chunk has a valid concept link and a page reference; every concept's prerequisites exist and the graph has no cycle.
- Every question has an answer key, a revision number and a misconception tag on each wrong option.
- Citations and chunk IDs resolve; glossary terms referenced in chunks exist.
- Total size is within the budget measured in Section 15, and the checksum verifies after upload.

Versions and stability. Concept and question IDs never change. A reworded question or changed key becomes a new revision; the old revision is marked legacy only after every supported app version has stopped serving it. Old packs remain usable offline, and the server interprets old events through the revision recorded in each event (Section 6).

Server retrieval (Decided): hybrid

- The query is normalised, and English technical terms are expanded through the glossary to their Odia forms and the reverse.
- Dense and keyword search each return the top 8; reciprocal rank fusion merges them and the top 4 chunks go to the generator.
- The embedding model is chosen by measurement: compare at least two multilingual models on 100 labelled Odia queries by recall at 5 and record the choice in Section 18.
- If the best score is below the threshold set from that evaluation, the agent does not answer from the model's own knowledge. It says the topic is outside the current lessons and offers the nearest concept.

Local retrieval on the phone (Decided). BM25 keyword ranking over the pack's Odia index plus glossary expansion. The answer is extractive: the best chunk, the key sentence highlighted, and a link to the related concept. No model is needed offline.

Grounding rule. Every generated factual answer must cite at least one chunk ID and the chapter reference. Section 13 defines the checks that enforce it.

12. Odia language and voice strategy

The language bonus is judged on whether the experience feels native, not on translated labels. Odia is a first-class requirement in every layer, owned by the Odia, assessment and evaluation lead.

What makes it native (Decided)

- Interface text written by a native speaker, not machine-translated: short sentences, everyday words, no jargon in navigation.
- Technical terms shown as learners meet them in class: the Odia term with the English term in brackets, driven by the glossary.
- Code-mixed input is expected: Odia script, Odia written in Latin letters, and Odia with English terms. The query normaliser handles all three.
- Answers use the register of a Class 9 textbook explanation, with one everyday example where the chunk allows.
- A bundled Odia font subset (for example Noto Sans Oriya) ships in the app shell, because low-end phones often render Odia badly.
- Odia and Latin numerals are both accepted in input; display follows a learner setting.

Text first, voice second (Decided). Text-first Odia doubts are the P0 path. Voice is P1, online only, with text as the fallback at every step. Offline learning never depends on ASR or TTS.

Capability	Approach	Fallback
Speech to text	Hosted Odia ASR through `/asr`; candidates to test include AI4Bharat models and Bhashini services	Typed input
Text to speech	Hosted Odia TTS through `/tts`; same candidate pool	Pre-generated pack audio, then text only
Offline voice	Not in the MVP	Text only

Which Odia ASR and TTS services work reliably is unverified. A timeboxed voice spike on real Odia recordings can run in parallel with any milestone; integration into the product starts only after Milestone 3 and only if the voice gate of Section 15 passes. Reliability and thresholds are set after the spike, not claimed before it.

Generation

- The generator receives retrieved Odia chunks and the glossary, with a prompt that fixes language, register, length limit and the citation rule. The prompt has a version and a hash recorded in every generation record.
- At least two LLMs are compared on the rubric below; the better one is the default and the other is the fallback behind the same provider interface.
- Authoring-time translation from English sources, if needed, uses IndicTrans2 or an equivalent and is always followed by human review.

Odia quality rubric (Default)

Dimension	Scored 1 to 5 by a native reviewer	Target
Correctness against the cited chunk	Facts match the source	4 or higher on 90% of samples
Fluency	Reads as natural Odia	4 or higher on 90%
Register	Right level for Class 9	4 or higher on 90%

Dimension	Scored 1 to 5 by a native reviewer	Target
Terminology	Glossary terms used consistently	95% of terms correct

The evaluation set and release gate are defined in Section 15.

13. Validation, safety and failure handling

Nothing reaches the learner or the database without passing an ordered validation contract, and every tool has a defined failure path.

Validation contract for generated answers (Decided). Checks run in this order and stop at the first failure. Each check is either deterministic or model-based, and the table says what happens when it fails.

Order	Check	Type	Pass condition	On failure
1	Schema	Deterministic	Output parses into the answer schema	Regenerate once, then extractive answer
2	Source references	Deterministic	At least one cited chunk ID, each exists in the retrieved set, and the chapter reference is present	Regenerate once, then extractive answer
3	Language and length	Deterministic	Predominantly Odia script, within the length cap, glossary terms used	Regenerate once, then extractive answer
4	Safety and educational fit	Deterministic plus model-based online	On-syllabus score above threshold, age-appropriate, no personal data; chunk text and learner input treated as data, never as instructions	Polite Odia redirect; event logged

Order	Check	Type	Pass condition	On failure
5	Grounding	Model-based (entailment by a second model call)	Every factual sentence is supported by the cited chunks	If the entailment check fails twice or is unavailable: do not show the generated text. Show a clearly labelled extractive answer, or refuse and redirect when no chunk is relevant enough

Keyword overlap is never accepted as proof that a factual claim is correct. It may be used only to choose which passage to show in an extractive answer.

Validation contract for decisions (Decided). Before a decision is saved or displayed, all of these are deterministic checks: the target concept exists in the installed graph, the action and reason code are in their enums, the delivery mode is feasible for the installed packs and network state, background actions are valid and sorted, and the ruleset version is known. Failure triggers the fallback decision of Section 6 and a high-severity log entry.

Tool failure matrix

Tool	Failure	Behaviour
Retriever (server)	Timeout or empty result	Retry once, then keyword-only search on the server index, then the local-style extractive path
LLM generator	Timeout, rate limit, bad output	Switch to the fallback provider; if that also fails, extractive answer marked basic
Entailment check	Unavailable	Extractive answer or redirect, as in check 5
ASR (P1)	Timeout or low confidence	Ask the learner to type or repeat; never guess silently
TTS (P1)	Failure	Show text only

Tool	Failure	Behaviour
Pack download	Interrupted	Resume from the last verified part; keep the old pack
Sync	Connection drops mid-upload	Events stay in the outbox; idempotent retry
Database write	Transaction fails	Whole batch RETRYABLE; nothing partially applied
Decision engine	Unexpected input or rule error	Fallback decision; high-severity trace

Observability (Decided). Structured JSON logs carry request_id, pseudonymous learner_id, decision_id and tool name. OpenTelemetry traces span API, orchestrator and tools. Metrics cover decision latency, LLM latency, fallback rate, sync lag, outbox depth, dead jobs and quarantined events. A developer panel on the phone shows the last 20 decisions and their traces.

Prompt injection. Retrieved chunks and learner input are delimited and labelled as untrusted data. The generator has no tools beyond the whitelisted retrieval calls, so injected instructions cannot trigger actions.

14. Tech stack, repository, security and release gates

Stack (Decided)

Layer	Choice	Notes
Client	React, TypeScript, Vite, Workbox PWA, IndexedDB via Dexie	Text-first, installable, Odia font bundled, seed pack in the app shell
API	FastAPI, Pydantic v2, Uvicorn	OpenAPI generated from models
Orchestration	LangGraph with PostgreSQL checkpointer	Doubt subgraph and decision graph
Data	PostgreSQL 16, FAISS index files, object storage (MinIO or S3), Redis	Redis is cache and limits only
Jobs	Arq, with a PostgreSQL job table as the source of truth	One framework only

Layer	Choice	Notes
AI services	Providers behind typed interfaces: LLM, embeddings, ASR, TTS, entailment	Chosen by measurement (Sections 11, 12)
Tooling	Docker, Docker Compose, GitHub Actions, pytest, Vitest, Playwright, Ruff, ESLint, Alembic	CI runs everything on each pull request

Repository layout (Decided)

```
saarthi/
  apps/
    web/            # React PWA
    api/            # FastAPI service
    worker/         # Arq jobs: packs, embeddings, indexes
  packages/
    contracts/      # Pydantic models -> OpenAPI -> TS types (generated, not
hand-edited)
    ruleset/        # rules.json, reason codes, shared test vectors
    decision-ts/    # client decision engine (pure functions)
  content/          # sources, glossary, questions, build scripts
  infra/            # docker-compose, CI, deploy config
  docs/             # this document, decisions.md
  tests/            # e2e, fault matrix, parity, replay
```

Contract change process (Decided). Contracts, the action enum, event types, reason codes, pack manifest and sync outcome semantics are frozen at the end of Milestone 1. After that, a change needs a pull request that updates the Pydantic model, regenerates OpenAPI and TypeScript types, updates the shared test vectors and passes both engines, with review from the agentic AI owner and the backend owner. A change in only one service fails CI.

Environments and deployment. Local: Docker Compose, one command. Staging: one VM running the same Compose file with real providers. Demo: staging plus backups and monitoring. Configuration comes from environment variables; secrets are never committed. For the hackathon the containers run on one cloud VM behind a reverse proxy with automatic TLS, written so the same images move to a managed container service without code changes.

Security and privacy (Decided)

- TLS everywhere, short-lived tokens, per-device rate limiting, request size limits, strict CORS.
- Learners are minors. Store only what the schema allows: pseudonymous ID, display name, language, learning events. No contacts, location or advertising identifiers. Voice audio is deleted after transcription unless a consent flow, once defined, says otherwise.
- The local profile PIN separates profiles on a shared phone and is not authentication (Sections 7 and 8).
- Secrets in the environment or a secret manager; dependency scanning in CI.

Release gates (Decided). These conditions are binding and are not waived for a deadline.

Gate	Condition	Until it passes
R1 Children's data	Consent, retention, deletion, access and incident-handling obligations for children's data are checked with appropriate guidance and documented	No pilot with real students; demo and testing use seeded data only
R2 Content rights	Permission to use and redistribute the textbook or source content is confirmed in writing	No learning pack is published; only team-authored or permitted content is used
R3 Odia quality	The Odia release gate of Section 15 passes for the release being shipped	No release to the demo or a pilot
R4 Offline integrity	The full fault matrix of Section 15 passes against the release build	No release
R5 Target device	Latency, memory and size targets are measured on the reference device and recorded	Numbers stay labelled as targets, not as achieved

UX rules for low digital confidence (Decided): one primary action per screen, large touch targets, icons with Odia labels, no typing needed for the main learning loop (answers by tapping), an audio prompt on each screen where pack audio exists, no more than three taps from Home to any learning action, and the next-action card always in the same place.

15. Testing, measurement and release gates

Numbers in this document are targets to verify. A target is reported as met only with a measurement on the reference device, recorded with its method.

Metrics (Decided, replaces the v1 explicit-action metric)

Metric	Target	How it is measured
Action validity	100%: every decision has one valid learning action, feasible delivery, valid reason code	Invariant checked on every scenario and in production traces; any violation fails CI
Decision quality	At least 90% agreement with independently labelled expected actions (target to be confirmed once the labelled set exists)	Gold scenario set below
Offline state preservation	100% of events preserved and equal to canonical replay	Fault matrix below
Engine parity	100% identical results from Python and TypeScript	Parity suite below
Odia quality	Rubric targets of Section 12	Odia release gate below
Local decision latency, launch time, memory, pack size	Targets of Section 3	Reference-device protocol below

An engine that always returns a valid action can score 100% on validity while choosing poor interventions, which is why quality is measured separately against expected actions.

Decision-quality gold set (Decided). At least 120 scenarios, each a normalised state and environment, labelled with the expected learning action and reason by two people who did not write the rules (the subject expert and the Odia lead), with disagreements resolved by discussion and recorded. At least 30% are held out and not used while tuning thresholds. Required coverage: prerequisite gaps, repeated wrong answers, no progress after remediation, mastered concepts, slice complete, missing pack while offline, low storage, weak network, cold start, and corrupt or stale state. Disagreements between the engine and the labels are investigated as either a rule fault or a labelling fault; thresholds are not tuned until the set merely passes.

Cross-language parity (Decided). Identical normalised inputs must give identical action, target concept, delivery mode, background actions, reason code and ruleset version in Python and TypeScript. The suite includes missing and invalid fields, out-of-range mastery, an unknown ruleset version (expected: typed error and the fallback decision) and BKT update sequences checked step by step in basis points.

Replay and property tests. Replaying an accepted event log in canonical order gives the same mastery and decision every time; shuffled delivery order, duplicates and late arrival of the same events give the same final projection.

Test layers

- Unit: planner rules, BKT, query normaliser, pack validator, outbox, quarantine.
- Integration: API with real PostgreSQL and a stubbed LLM; sync outcomes; pack download resume; job retry and dead-letter.
- End to end: Playwright with throttling and offline toggling through the full learner loop.
- Profile isolation: switch profiles while offline and prove one learner can never read or change another's mastery, outbox, decisions, session state or cached UI.
- Language: the Odia set and the voice set below.

Fault matrix (all P0; each result compared with a canonical replay of the accepted log)

Case	Expected result
App killed during the local transaction	No partial event; state and outbox agree
Airplane mode during an assessment, then reconnect	All events on the server once; mastery equals replay
Duplicate sync requests	DUPLICATE outcomes; no double effect
Partial server acceptance	Only accepted events leave the outbox; others stay or are quarantined
Permanent rejection	Event quarantined, later events unaffected, visible in the developer panel
Connection drops mid-sync	No duplicates; remainder sent next time
Expired access and refresh tokens after a long offline period	Learning continues; renewal by signed challenge; outbox intact
Device clock set a day ahead	No change in ordering or mastery

Case	Expected result
Storage full	Clear Odia warning; no data loss; installed packs still usable
Corrupted pack	Checksum failure detected; old pack kept or re-download prompted
Pack upgraded while offline, old question revisions in events	Accepted with correct effect or effect none; mastery not corrupted
Outdated or unknown ruleset	Fallback decision; no crash; server decision wins on sync
Profile switch while offline	Isolation holds; each profile syncs its own events
Sync failing repeatedly	Local learning continues without error

Reference device and measurement protocol (Decided). Before the checkpoint the team records in `docs/devices.md` the exact reference device (an Android phone with 2 GB RAM or less), its Android version, the browser and its version, the three-chapter pack used, and the network profile (a throttled profile of about 400 kbps and 400 ms RTT in browser tools, plus at least one run on a real weak mobile connection). Launch time, decision latency and memory are each measured over 10 runs and reported as median and p95, with the method named. If the installed app plus one pack exceeds 25 MB on that device, the pack strategy is revised from the measurement (smaller per-chapter packs, no audio in the base pack, stronger compression) and the target is changed in this document openly.

Odia release gate (Decided, R3). The 100-question learner-style set is re-scored by the native reviewer before every release, with a record of the provider and model version, prompt version, scores, sample sizes and any regression compared with the last release. The set covers Odia script, Latin-script Odia, code-mixed Odia and English, textbook terminology, Unicode normalisation and both numeral systems. A release that misses a rubric target does not ship.

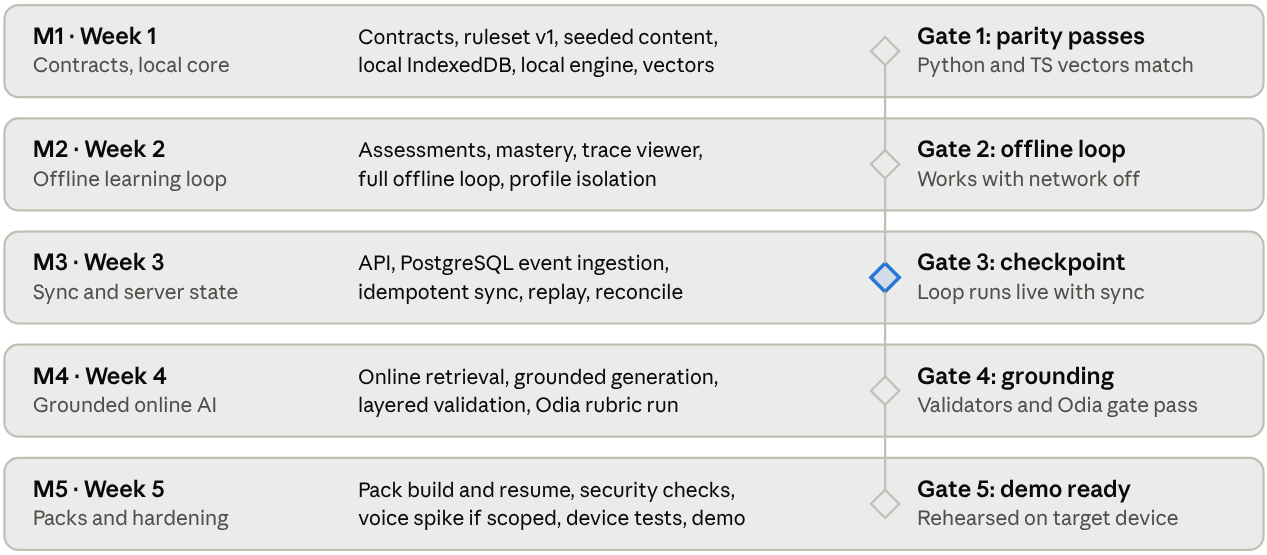
Voice gate (P1). ASR and TTS thresholds are set after the real-audio spike of Section 12 and recorded here; voice is integrated and demoed only if the recorded set meets them.

Release gate summary. A build goes to the demo environment only when CI is green, action validity is 100%, gold-set agreement and parity are at target, the whole fault matrix passes, the Odia gate passes, and the reference-device measurements are recorded.

16. Roadmap, ownership and definition of done

The milestones follow dependency order: the offline core is built and tested before anything depends on remote AI. Each gate is a test result. This replaces the five-week plan in the pitch deck, whose implementation slide and success-metric slide should be updated to match.

Five milestones in dependency order; each gate is a test result, not a demo



roadmap · five milestones with test gates

The date of the mid-evaluation checkpoint is not yet known; Gate 3 assumes the end of Week 3 until the organisers confirm (Section 18, D4).

Milestone deliverables and acceptance (Decided)

Milestone	Scope	Must be true at its gate
M1	Contracts, rule table (rules.json v1), seeded content for the three chapters, local IndexedDB writes with outbox, local decision engine, shared test vectors	Contracts and event, action, reason-code, sync-outcome and pack formats frozen; Python and TypeScript engines pass identical vectors, including invalid input
M2	Assessments, mastery updates, trace viewer, the complete offline loop, profile isolation	A learner completes the full loop with the network off; profile-isolation tests pass; gold-set labelling started

Milestone	Scope	Must be true at its gate
M3	API, PostgreSQL event ingestion, idempotent sync with per-event outcomes, server replay, client reconciliation, auth renewal	Core fault matrix cases pass; server mastery equals canonical replay; running agent loop with real tool calls and traces shown at the checkpoint
M4	Grounded online retrieval and generation, layered validation, provider fallbacks, Odia evaluation run	Validation contract enforced; extractive fallback proven by forced provider failures; Odia gate run recorded
M5	Pack build, download and resume, security checks, voice spike and integration if in scope, low-end device tests, demo rehearsal	Full fault matrix passes; reference-device measurements recorded; demo rehearsed three times on the target device

Work in each milestone is merged only through the Definition of Done below. A component that merely starts does not close a milestone.

Ownership (from the pitch deck, Decided)

Owner	Area	Primary deliverables
Shuvakanta Patra	Agentic AI and decision engine	LangGraph graph, rules.json, reason codes, shared test vectors, TypeScript decision engine, traces, gold-set tooling
Shreyans Behra	Backend, RAG, data	FastAPI service, PostgreSQL schema and migrations, ingestion and replay, retrieval, pack builder, jobs, deployment, backups
Sadhna Sathpati	Frontend and offline systems	React PWA, ProfileSession store, outbox and quarantine, service worker, network monitor, auth renewal, pack downloads, UX rules
Subodh Kumar Swain	Odia, assessment, evaluation	Odia interface text, glossary, content curation, question bank, gold-set labelling, Odia rubric scoring, voice spike

Shared ownership: the contracts package belongs jointly to the agentic AI owner and the backend owner, and changes need both reviews.

Definition of done for any task (Decided)

- [ ] Code reviewed by one other owner and merged to main through CI
- [ ] Unit and integration tests written and passing; shared test vectors updated if rules changed
- [ ] Typed errors handled per Section 13 and trace entries emitted
- [ ] Offline behaviour implemented and covered by the fault matrix where relevant
- [ ] Contracts regenerated and this document updated if a decision changed
- [ ] All learner-facing text present in Odia and reviewed by the Odia owner
- [ ] Verified on the reference device where it affects performance, size or offline behaviour

The system is not called production-ready because backups, monitoring, TLS and retries are specified. It is called production-minded until the operational tests have been run and the remaining limitations are written down.

17. Live demo script and judging checklist

The organisers check a live end-to-end flow with real tool calls. The demo runs against the real build on the reference phone, with seeded data for one learner (Sunita) who already has partial mastery in two concepts. The core flow is text-first and needs no voice; the voice step is added only if the voice gate of Section 15 has passed.

Demo flow (Default, about 7 minutes; adjust to the slot we are given)

Step	What we do	What the judges see
1	One sentence: who the learner is and the gap	Problem understanding
2	Open the installed PWA on the phone, Odia interface; the next-action card is waiting	Ease of use, first-time flow
3	Type a doubt in Odia; optionally speak it if voice passed its gate	Odia input, grounded Odia answer with chapter reference
4	Take a 3-question check, answer one wrongly	Mastery update, misconception tag

Step	What we do	What the judges see
5	Open the decision trace panel	Tools called, order, reason code, ruleset version
6	Switch the phone to airplane mode live	Network badge changes to offline
7	Continue: read a cached chunk, answer more questions, ask a doubt and get a basic pack-based answer, receive a local decision	Offline continuity; the answer is honestly labelled basic
8	Reconnect	Outbox drains with per-event outcomes; the new suggestion appears at the next natural break; mastery matches the server
9	Show local and server decisions side by side in the trace	State continuity, deterministic core

Judging checklist

- [ ] Problem and user stated in one sentence
- [ ] Real tool calls, no static screens
- [ ] Decision trace shows tool order and reasons
- [ ] Interaction in Odia (text; voice only if it passed its gate)
- [ ] A first-time user completes the main task unguided (invite a judge or volunteer)
- [ ] Network switched off live, fallback works, data syncs back
- [ ] Clear answer on deterministic versus autonomous

Prepared answers

Question	Short answer
Who is the user and what changes?	A Class 9 Odia-medium learner on a shared low-end phone; doubts answered in Odia and a next step chosen for her, even offline
Why deterministic, and what breaks otherwise?	Offline operation and auditability; an autonomous planner needs a cloud model and varies per run (Section 4)

Question	Short answer
Which tools, and what if one fails?	Retriever, generator, validators, pack service, sync, decision engine; each has a fallback (Sections 5, 13)
What happens when the network drops mid-task?	State and event are saved in one local transaction, the local engine decides, sync is idempotent with per-event outcomes (Sections 8, 9)
How is language more than translation?	Native-written interface, class glossary, code-mixed input, rubric-scored Odia answers (Section 12)
Is it production-ready?	It is a production-minded MVP; what has and has not been verified is listed in Sections 15 and 18

Backup plan: a recorded run of the full flow on the same build, a second phone with the same seeded data, and a local Compose stack that works with no internet if the venue network fails.

18. Risks, open decisions and glossary

Risks and mitigations

Risk	Likelihood and impact	Mitigation
LLM gives wrong or poor Odia	High, high	Grounded retrieval, ordered validation, extractive fallback, native reviewer, Odia release gate
Odia ASR or TTS unreliable	High, medium	Voice is P1 and online only; text path is complete without it; spike before integration
Sync bugs lose or duplicate events	Medium, high	Event sourcing, per-event outcomes, idempotent keys, replay oracle, fault matrix in CI

Risk	Likelihood and impact	Mitigation
Mastery diverges between phone and server	Medium, high	Integer basis points, shared rules.json, parity suite
Content creation slower than code	High, high	Three chapters only, content work from Week 1, a named content owner
Low-end device cannot cope	Medium, medium	Text-first, small packs, early reference-device measurement
Rights or children's-data issues block a pilot	Medium, high	Release gates R1 and R2; demo uses seeded and permitted content
Learner needs two phones	Low for the MVP, medium later	Explicitly out of scope; second device rejected; merge policy deferred
Venue network fails during the demo	Medium, high	Local stack and recorded backup
Scope creep from stretch features	High, medium	P2 items start only after all P0 tests pass

Open decisions (the owner decides, then this table is updated)

#	Decision	Default if undecided	Owner
D1	Rights to use and redistribute the Class 9 Science textbook text	Use only content with written permission; otherwise write original concept notes from the syllabus	Odia and evaluation lead
D2	Which three chapters form the MVP slice	The three whose concepts are best explained in text alone	Team
D3	Which LLM, embedding model, ASR and TTS	Decided by the measurements in Sections 11, 12 and 15	Agentic AI lead and Odia lead
D4	Date and format of the mid-evaluation checkpoint	Treat the end of Week 3 as the target	Team

#	Decision	Default if undecided	Owner
D5	Children's data obligations before any pilot	No real-student pilot; seeded data only (gate R1)	Team with faculty advice
D6	Hosting provider and budget for cloud LLM and ASR use	One VM plus pay-per-use APIs with a hard monthly cap	Backend lead
D7	On-device model for offline doubts	No; extractive answers only	Agentic AI lead
D8	The reference phone, browser and network profile	The cheapest 2 GB Android phone the team can obtain, current stable Chrome, recorded in docs/devices.md	Frontend lead
D9	Multi-device learners	Out of scope for the MVP	Team
D10	Voice in the demo	Include only if the voice gate passes	Odia lead

Known limitations to state openly: no multi-device learners; local data is not encrypted at rest; re-linking a learner to a new phone after losing the device key is not supported; offline doubts are extractive, not generated; Odia voice is unverified until the spike; all performance numbers are targets until measured.

Glossary

Term	Meaning
Learning action	One of Remediate, Retry, Revise, Advance; returned with a delivery mode and any background actions (Sync, Prefetch)
Mastery	Estimated probability that a learner knows a concept, stored in basis points (0 to 10000)
Basis point	One hundredth of a percent; mastery is stored as an integer so every platform computes the same decision
Pack	A versioned, checksummed bundle of chunks, questions and glossary for offline use

Term	Meaning
Outbox	The phone's queue of events not yet acknowledged by the server
Quarantine	Phone-side store for permanently rejected events, kept for diagnosis and never silently dropped
Ruleset	Versioned rules.json shared by the server and phone engines
Extractive answer	An answer made by selecting text from the curriculum rather than generating new text
Trace	Stored record of inputs, tools called and the reason behind each decision
Canonical replay	Rebuilding mastery by applying accepted events in seq order; the oracle for correctness

Appendix A. Ruleset v1 and reason codes

The file `packages/ruleset/rules.json` is loaded by both the Python and TypeScript engines. Values are starting defaults, tuned only against the held-out gold set of Section 15. Any change bumps `version` (and `bkt_version` if the mastery update changes) and updates the shared test vectors; a `bkt_version` change forces a full projection rebuild by replay.

```
{
  "version": "1.0.0",
  "bkt_version": "1",
  "bkt": { "p_init_bp": 2000, "learn": 0.15, "slip": 0.10, "guess": 0.20,
           "clamp_bp": [100, 9900], "rounding": "half_up" },
  "bands": { "weak_below_bp": 4000, "mastered_at_bp": 8500 },
  "guard": { "attempts_without_gain": 3, "min_gain_bp": 200 },
  "network": {
    "online": { "rtt_ms_max": 800,  "downlink_kbps_min": 400 },
    "weak":   { "rtt_ms_max": 3000, "downlink_kbps_min": 50 },
    "probe_interval_s": 30, "probe_timeout_s": 3
  },
  "prefetch": { "min_free_mb": 150, "battery_saver_blocks": true },
  "sync": { "batch_size": 100, "backoff_s": [5, 15, 60, 300] },
  "doubt": { "max_tool_calls": 3, "budget_s": 12, "top_k": 4 },
  "auth": { "access_ttl_h": 24, "refresh_ttl_d": 30 }
}
```

Enumerations (frozen at the end of Milestone 1)

- Learning actions: REMEDIATE, RETRY, REVISE, ADVANCE
- Background actions: SYNC, PREFETCH
- Delivery modes: cached, rich, prefetch_then_offline
- Sync outcomes: ACCEPTED, DUPLICATE, RETRYABLE, REJECTED

Reason codes (stable; never renamed, only added)

Code	Action	Meaning
STATE_INVALID_FALLBACK	RETRY or ADVANCE	State or ruleset invalid; safe fallback used
COLD_START	ADVANCE	New learner; first concept without prerequisites
WEAK_PREREQ_GAP	REMEDIATE	Weak concept with a weak prerequisite
WEAK_CONCEPT	REMEDIATE	Mastery below the weak threshold
WRONG_RETRY	RETRY	Partial mastery, last answer wrong

Code	Action	Meaning
PARTIAL_PRACTICE	RETRY	Partial mastery, last answer correct
MASTERED_ADVANCE	ADVANCE	Mastered; next eligible concept
NO_GAIN_ALT_CONTENT	REVISE	No gain after repeated attempts; alternate content format
NO_GAIN_REMEDIATE_PREREQ	REMEDIATE	No gain; weakest prerequisite first
INTERVENTIONS_EXHAUSTED	REVISE	Terminal; needs_help set, no advancement
SLICE_COMPLETE	REVISE	Every installed concept mastered; spaced review
BLOCKED_BY_PREREQ	REMEDIATE	No eligible next concept; unmet prerequisite
CONTENT_NOT_AVAILABLE_OFFLINE	Same type as intended	Target pack missing offline; nearest installed concept used

Appendix B. Screens

Eight screens cover the MVP. Each has one primary action and works offline unless marked.

Screen	Primary action	Notes
Profile picker	Tap a name to start	Optional local PIN; add profile; each profile opens its own database
Home	Tap the next-action card	One-line Odia reason, network badge, sync state; pending suggestion appears only at natural breaks

Screen	Primary action	Notes
Learn	Read a chunk, tap Next	Glossary terms tappable; audio button only if the pack has audio
Practice	Tap an answer	MCQ first; instant Odia feedback; a skip is recorded as QUESTION_ABANDONED
Ask a doubt	Type a doubt	Online: grounded generated answer with chapter reference. Offline: pack-based answer labelled basic. Voice button (P1) shown only when online and enabled
Progress	View the concept map by colour	Weak, partial, mastered; no scores or rankings; an Odia prompt to ask a teacher when needs_help is set
Downloads	Install or update a pack	Size shown, resumable, storage warning
Developer panel	View last 20 decisions, quarantine, dead jobs	Hidden behind a long press; used in the demo

Appendix C. Brief to give a coding agent

Paste this with the full document attached.

```
You are building SAARTHI as a production-minded MVP, following the attached
design document exactly.
1. Read Sections 1 to 9 and Appendix A before writing code.
2. Work in the milestone order of Section 16, one milestone per branch, and
stop at each gate to run the tests of Section 15 and report pass or fail.
3. Create the repository layout of Section 14. Define the Pydantic contracts
first and generate OpenAPI and TypeScript types from them. Never hand-edit
generated types.
4. Implement the decision engine as pure functions in Python and TypeScript,
both reading rules.json, storing mastery as integer basis points, and passing
the same test vectors, including invalid input.
5. Never let an LLM choose the learning action. Never call the network or an
LLM from the local engine.
6. Every tool and provider call returns a typed result or a typed error
(Section 5). Providers never write to a database.
7. Events are written with their state change in one local transaction. Sync
returns one outcome per event; remove an event from the outbox only on
ACCEPTED, DUPLICATE or after moving it to quarantine.
8. Keep profile data isolated through one ProfileSession module. Do not cache
API responses that contain learner data.
9. Generated answers pass the ordered validation contract of Section 13; if
grounding cannot be verified, show a labelled extractive answer or redirect.
10. If the document is silent, take the stated default, record it in
docs/decisions.md, and continue. Do not add features outside Section 2 or
infrastructure outside Section 14.
11. Report what you verified and what you did not. Do not describe the system
as production-ready.
```

The human owners keep the content, model and voice decisions of Sections 11 and 12, because they depend on measurement and Odia review, not on code.